

Comunicação, Desempenho e Segurança Avançada

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Laboratório Prático: Domínio da Rede Restrita

Contexto do Lab: Imagine que é um estudante na Universidade. Está ligado à rede **eduroam**, uma rede que é segura mas muito restritiva. Quer correr um projeto onde um sensor em sua casa sincroniza dados para um servidor, e precisa de aceder a uma base de dados privada a partir do seu portátil enquanto está na biblioteca da Universidade. As ligações diretas estão bloqueadas. Deve usar as suas competências para **medir**, **sincronizar** e criar **túneis** através destas limitações.

Parte 0: Configuração e Infraestrutura

Objetivo de Aprendizagem: Implementar uma simulação de “Rede Empresarial Restrita” usando contentores.

Utilizaremos o Docker para criar o nosso mini-ecossistema universitário.

1. Criar a Rede do Laboratório

```
$ docker network create --subnet=172.25.0.0/16 lab-net
```

2. Lançar a Infraestrutura

Crie uma pasta `class08-lab` e um ficheiro `compose.yml`:

```
services:
  # Representa o seu Servidor em Casa (o destino para backups)
  home-server:
    build:
      context: ./home-server
    container_name: home-server
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Lisbon
      - USER_NAME=student
      - PASSWORD_ACCESS=true
      - USER_PASSWORD=studentpass
    ports:
      - "2222:2222"

  networks:
    lab-net:
      ipv4_address: 172.25.0.10

# Representa o seu Portátil (ligado à eduroam restrita)
laptop:
  image: alpine:latest
```

```

    container_name: laptop
    networks:
      lab-net:
        ipv4_address: 172.25.0.20
    command: sh -c "apk add rsync openssh-client iperf3 mtr tcpdump bash curl sshpass && s

# Representa um alvo de tráfego remoto na Internet
internet-target:
  image: alpine:latest
  container_name: internet-target
  networks:
    lab-net:
      ipv4_address: 172.25.0.30
  command: sh -c "apk add iperf3 && iperf3 -s"

# Representa uma Base de Dados Privada (isolada de todos)
private-db:
  image: alpine:latest
  container_name: private-db
  networks:
    lab-net:
      ipv4_address: 172.25.0.40
  command: sh -c "apk add python3 && echo 'DADOS_SENSIVEIS_DE_ESTUDANTE' > secret.txt &&

# Representa uma Gateway para Serviços da Universidade
uni-gateway:
  image: nginx:alpine
  container_name: uni-gateway
  ports:
    - "8081:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
  networks:
    lab-net:
      ipv4_address: 172.25.0.100

networks:
  lab-net:
    external: true

```

3. Criar os Ficheiros de Configuração Crie uma subpasta home-server e um Dockerfile dentro dela:

FROM lscr.io/linuxserver/openssh-server:latest

RUN apk add --no-cache rsync

Crie um ficheiro nginx.conf na pasta principal class08-lab:

```

events {
    worker_connections 1024;
}

http {
    upstream university_services {
        server 172.25.0.40:5432;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://university_services;

```

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

4. Iniciar o Ambiente

```
$ docker compose up -d
```

Parte 1: Desempenho e Monitorização

Objetivo: Provar porque é que uma rede parece “lenta” usando dados.

Exercício 1: Teste de Débito (iperf3) Cenário Real: Está a tentar descarregar um conjunto de dados grande na biblioteca, mas está a demorar uma eternidade. É do servidor ou do Wi-Fi?

1. Execute o cliente a partir do seu laptop para o internet-target:

```
$ docker exec -it laptop iperf3 -c 172.25.0.30
```

2. **Análise:** Se o débito for 1Gbps, a rede está perfeita. Se for 1Mbps, encontrou o estrangulamento.

Exercício 2: Análise em Tempo Real (mtr) Cenário Real: A sua videochamada está sempre a cair. Suspeita de um router defeituoso no edifício da Universidade.

1. Execute o mtr do seu laptop para o home-server:

```
$ docker exec -it laptop mtr 172.25.0.10
```

2. Olhe para a coluna de perda (loss). Se a perda começar no primeiro salto, o Access Point local está com problemas.
-

Parte 2: Sincronização Eficiente (rsync)

Objetivo: Mover dados sem desperdiçar tempo ou largura de banda.

Exercício 3: Backups Inteligentes Cenário Real: Tem um ficheiro de projeto de 100MB. Apenas mudou um parágrafo. Na eduroam, a largura de banda de upload é limitada.

1. Crie um ficheiro grande no seu laptop: `docker exec -it laptop dd if=/dev/urandom of=projeto.pdf bs=1M count=10`.

2. Sincronize para o seu home-server (pass studentpass):

```
$ docker exec -it laptop rsync \
  -avz -e 'ssh -p 2222' /projeto.pdf student@172.25.0.10:/config/
```

3. Modifique ligeiramente o ficheiro e sincronize novamente. Note quão mais rápido é.
-

Parte 3: Contornar Limitações (SSH)

Objetivo: Alcançar o que está escondido ou bloqueado.

Exercício 4: Local Port Forwarding Real-world Scenario: Precisa de consultar a private-db para a sua tese, mas a eduroam bloqueia a porta 5432. Tem acesso SSH ao seu home-server.

1. Abra o túnel a partir da sua **máquina real** (host): `ssh -p 2222 -L 9000:172.25.0.40:5432 student@localhost`.
2. Aceda a `http://localhost:9000` na sua máquina. Conseguiu com sucesso estabelecer uma “ponte” para a BD isolada.

Exercício 5: Proxy SOCKS Dinâmico Real-world Scenario: A Universidade bloqueia um site de investigação específico que você precisa. Usa a sua ligação de casa para navegar através dela.

1. Crie o proxy: `ssh -p 2222 -D 1080 student@localhost`.
 2. Configure o curl do seu portátil para o usar:
`$ curl --proxy socks5h://localhost:1080 http://172.25.0.40:5432`
-

Parte 4: Fiabilidade Avançada

Exercício 6: Equilíbrio de Carga (Load Balancing) Cenário Real: Milhares de estudantes acedem ao portal de notas ao mesmo tempo.

1. Use a uni-gateway para distribuir o tráfego e teste o que acontece se um servidor ficar offline.